

Design Document

Smart Library

ระบบห้องสมุดอัจฉริยะบนบอร์ด FPGA

จัดทำโดย

นายณัฐภูมิ บาดขุนทด	รหัสนักศึกษา 68010366
นางสาวปริญา บุญสิงมา	รหัสนักศึกษา 68010662
นายพงศธร ฟักผล	รหัสนักศึกษา 68010726
นางสาวปลิตา ชูรัตน์	รหัสนักศึกษา 68010669
นางสาวฑริตา เปรมชุติวัต	รหัสนักศึกษา 68010283
นางสาวณัฐกฤตา ลิ้มธีระวัฒน์	รหัสนักศึกษา 68010324

วิชา

Digital System Fundamentals

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

สารบัญ

บทนำ 5

1. ที่มาและแนวคิด 5

2. วัตถุประสงค์ของโครงงาน 5

3. ประโยชน์และการประยุกต์ใช้ 5

4. ความสามารถของระบบ 6

5. วีดีทัศน์แนะนำชิ้นงาน 6

6. ข้อกำหนดของงานและการออกแบบให้สอดคล้อง 6

การศึกษาข้อมูลและทฤษฎีที่เกี่ยวข้อง 8

1. แหล่งข้อมูลที่ใช้ 8

2. การทดลองยืนยันข้อมูลบนบอร์ด 9

3. ข้อกำหนดการออกแบบที่ได้จากการหาข้อมูล 10

4. ทฤษฎีที่เกี่ยวข้อง 10

อุปกรณ์และเครื่องมือ 12

1. อุปกรณ์ 12

2. เครื่องมือและซอฟต์แวร์ 12

การออกแบบและการพัฒนา 13

1. แนวคิดการออกแบบแบบ Top-Down 13

2. Level 0: ระดับระบบ 14

3. Level 1: ระดับบอร์ด 15

4. Level 2–3: ระดับโมดูล 18

5. แผนภาพสถานะ (FSM) 22

6. การออกแบบโปรโตคอลสื่อสาร 22

7. การออกแบบส่วน ESP8266 และเว็บ 23
8. การออกแบบวงจรภายนอก 24
9. ลำดับการพัฒนา 24
10. ขั้นตอนการ build และอัปโหลด 25
11. ปัญหาที่พบและการแก้ไข 26
12. ทรัพยากรที่ใช้บน FPGA 26
โค้ด VHDL 28
1. uart_rx (ตัวรับ UART) 28
2. uart_frame_tx (ตัวส่งเฟรม UART) 29
3. debounce (กรองสัญญาณปุ่ม) 30
4. melody (เพลงตอน Follow) 31
5. ตัวถอดรหัสไฟโซนและไฟชั้น (board2_top) 33
ผลการทดลองและการทดสอบ 35
1. แนวทางการทดสอบ 35
2. ผลการทดสอบด้วยเครื่องมือ 35
3. ผลการทดสอบบนบอร์ดที่ยืนยันแล้ว 35
4. ตารางทดสอบรวมระบบ 36
5. ค่าที่ต้องปรับเทียบก่อนนำเสนอ 37
สรุปและข้อเสนอแนะ 38
1. สิ่งที่ประสบความสำเร็จ 38
2. ข้อจำกัดและสิ่งที่ยังต้องปรับปรุง 38
3. ข้อเสนอแนะ 38
แหล่งจัดเก็บข้อมูล 39

ภาคผนวก ก ตารางขาบอร์ด 1 และบอร์ด 2 40

ภาคผนวก ข รายการหนังสือและไฟล์ในโครงงาน 42

บทนำ

1. ที่มาและแนวคิด

Smart Library คือระบบห้องสมุดอัจฉริยะที่ใช้บอร์ด FPGA EDGE Spartan-7 (XC7S15-FTGB196) จำนวน 2 บอร์ดเป็นหน่วยประมวลผลหลัก ระบบควบคุมสภาพแวดล้อมของห้อง (ไฟ แอร์ พัดลม) เฝ้าระวังควัน แสดงจำนวนที่นั่งว่าง และช่วยผู้ใช้หาหนังสือด้วยไฟที่โซนและชั้นของหนังสือ ผู้ใช้ค้นหาหนังสือผ่านคอมพิวเตอร์ของห้องสมุดที่เชื่อมต่อ Wi-Fi ของโมดูล ESP8266 บนบอร์ด 1

แนวคิดหลักคือให้ตรรกะควบคุมทั้งหมดทำงานเป็นวงจรดิจิทัลบน FPGA ทุกระบบทำงานขนานกันจริงในระดับฮาร์ดแวร์ โดยไม่มีระบบใดต้องรอระบบอื่น ส่วน ESP8266 ทำหน้าที่เพียงเว็บเซิร์ฟเวอร์และตัวแปลงคำสั่งจากเว็บเป็นข้อมูลอนุกรม

2. วัตถุประสงค์ของโครงงาน

1. เพื่อออกแบบและพัฒนาระบบห้องสมุดอัจฉริยะ โดยใช้บอร์ด FPGA 2 บอร์ดเป็นหน่วยประมวลผลหลักในการควบคุมระบบทั้งหมดของห้องสมุด
2. เพื่อพัฒนาระบบควบคุมสภาพแวดล้อมในห้องสมุด ให้ไฟห้องเปิด-ปิดตามความสว่าง และแอร์ปรับอุณหภูมิตามอุณหภูมิห้องได้โดยอัตโนมัติในช่วง 16–27 °C
3. เพื่อเพิ่มความปลอดภัยด้วยระบบตรวจจับควันที่ส่งเสียงเตือนและแจ้งเตือนบนเว็บทันทีเมื่อเกิดอันตราย
4. เพื่ออำนวยความสะดวกแก่ผู้ใช้ในการดูที่นั่งว่างและค้นหาหนังสือ พร้อมไฟแสดงตำแหน่งโซนและชั้นของหนังสือ
5. เพื่อฝึกออกแบบวงจรดิจิทัลแบบ Top-Down และประยุกต์ความรู้ในรายวิชา Digital System Fundamentals

3. ประโยชน์และการประยุกต์ใช้

1. **ผู้ใช้ห้องสมุด:** หาหนังสือได้เร็วขึ้นเพราะเห็นไฟที่โซนและชั้นของหนังสือ และรู้จำนวนที่นั่งว่างก่อนเดินเข้าไป
2. **ผู้ดูแลห้องสมุด:** ไฟและแอร์ทำงานเองตามสภาพแวดล้อม และมีการเตือนควันทั้งที่บอร์ดและบนเว็บ ลดภาระการเฝ้าดู

3. **การประหยัดพลังงาน:** ไฟไม่เปิดทิ้งตอนกลางวัน และแอร์ไม่ทำงานเกินช่วงอุณหภูมิที่กำหนด
4. **การเรียนรู้:** ได้ฝึกออกแบบ FSM ตัวนับ การสื่อสาร SPI และ UART และการแบ่งงานระหว่าง FPGA กับไมโครคอนโทรลเลอร์ ซึ่งนำไปต่อยอดกับระบบอาคารอัจฉริยะอื่นได้

4. ความสามารถของระบบ

ตารางที่ 1 ระบบย่อยทั้งหมดของ Smart Library

ระบบ	อินพุต	เอาต์พุต	ควบคุมจากเว็บ
ไฟห้อง	LDR, ปุ่ม SW14, ปุ่ม SW18	LED ขาว 3 ดวง (ขาเดียว)	เลือกโหมด, เปิด/ปิด (Manual)
แอร์	LM35	7-segment	แสดงค่าอย่างเดียว
พัดลม	ปุ่ม SW16	พัดลม DC ผ่านวงจรขับ	เปิด/ปิด
เตือนควัน	MQ-2 (DO)	buzzer, LCD	สถานะ + แจ้งเตือนเต็มจอ
ที่นั่ง	สวิตช์ SW2 SW4 SW5 (ฐานสอง)	LCD 16×2	แถบที่นั่ง 7 ช่อง
หาหนังสือ	เว็บ (ปุ่ม Follow)	ไฟโซน + ไฟชั้น (บอร์ด 2), เพลง	ค้นหา + Follow

5. วีดีทัศน์แนะนำชิ้นงาน

ชื่อวีดีทัศน์: CE_KMITL_Digital_2569_SmartLibrary

ลิงก์ย่อ: [ใส่ลิงก์ย่อหลังอัปโหลดวีดีทัศน์]

6. ข้อกำหนดของงานและการออกแบบให้สอดคล้อง

ตารางที่ 2 ข้อกำหนดของงานกับแนวทางที่ใช้

ข้อกำหนด (Assignment 2569)	การออกแบบของ Smart Library
ใช้บอร์ด FPGA อย่างน้อย 2 บอร์ด	บอร์ด 1 ควบคุมห้อง บอร์ด 2 ควบคุมไฟโซนและไฟชั้น สื่อสารกันแบบ UART
ใช้ Microcontroller ประกอบได้ แต่เนื้อหาหลักต้องอยู่ใน FPGA	ESP8266 เป็นเว็บเซิร์ฟเวอร์และแปลงคำสั่งเท่านั้น การตัดสินใจทุกอย่าง (โหมดไฟ แอร์ เตือนควัน ที่นั่ง เพลง ไฟชั้น) อยู่ใน VHDL

ข้อกำหนด (Assignment 2569)	การออกแบบของ Smart Library
ประยุกต์ความรู้ในรายวิชา	FSM, ตัวนับ/ตัวแบ่งความถี่, decoder, วงจรบวกลบ, multiplex 7-segment, LCD, SPI, UART, debounce
ออกแบบส่วน FPGA แบบ Top-Down	แบ่งจากระบบ ⟶ บอร์ด ⟶ ระบบย่อย ⟶ entity ของ VHDL (บทการออกแบบและการพัฒนา)

การศึกษาข้อมูลและทฤษฎีที่เกี่ยวข้อง

ก่อนออกแบบ กลุ่มศึกษาข้อมูลจากเอกสารของบอร์ดและอุปกรณ์ แล้วทดลองบนบอร์ดจริงเพื่อยืนยันข้อมูลที่ไม่แน่ใจ เพราะพบว่าคู่มือบอร์ดบางส่วนไม่ตรงกับบอร์ดรุ่น Spartan-7 ที่ใช้จริง ข้อมูลที่ได้ถูกแปลงเป็นข้อกำหนดของการออกแบบในหัวข้อที่ 3

1. แหล่งข้อมูลที่ใช้

ตารางที่ 3 แหล่งข้อมูลและการนำไปใช้

แหล่งข้อมูล	ข้อมูลที่ได้	นำไปใช้ในการออกแบบ
คู่มือ EDGE Spartan 7 FPGA Development Board	ตำแหน่งขาอุปกรณ์บนบอร์ด, jumper J4 J6 J9 J11, buzzer แบบ piezo ผ่านทรานซิสเตอร์ (เรโซแนนซ์ 3.8 kHz), ESP8266 ต่อ FPGA แบบ UART, การโปรแกรม JTAG หายเมื่อไฟดับ	ตารางขา (XDC), เงื่อนไข jumper, การเขียน Flash เพื่อใช้งานถาวร
Datasheet MCP3208	ADC 12 บิต 8 ช่อง, SPI, ลำดับบิต start/SGL/D2-D0, ความเร็วนาฬิกาสูงสุดราว 1 MHz ที่ 2.7 V	ออกแบบ entity mcp3208 ที่ SCK 500 kHz อ่าน CH6 (LDR) และ CH7 (LM35)
Datasheet LM35	ให้แรงดัน 10 mV/°C	สูตรแปลงอุณหภูมิ
เอกสารโมดูล MQ-2	ขา DO เป็นดิจิทัล LOW เมื่อเจอควัน ปรับเกณฑ์ด้วยตัวต้านทานปรับค่า	อ่านขา DO ผ่าน debounce ไม่ต้องใช้ ADC
Datasheet จอ LCD (HD44780)	ชุดคำสั่งเริ่มต้น 0x38 0x0C 0x01 0x06, เวลารอคำสั่ง	FSM ของ lcd16x2 และเวลารอ 5 ms / 50 µs
ESP8266 Arduino Core 3.1.2	WiFi.softAP, ESP8266WebServer, รองรับเครื่องเชื่อมต่อสูงสุด 8 เครื่อง	ส่วนเว็บและ API
เนื้อหาและการทดลองในรายวิชา	FSM, ตัวนับ, decoder, 7-segment, LCD, ADC, UART	โครงสร้างของแต่ละ entity

2. การทดลองยืนยันข้อมูลบนบอร์ด

ข้อมูลบางส่วนในคู่มือขัดแย้งกับไฟล์ข้อกำหนดขาทางการ จึงทดลองบนบอร์ดก่อนออกแบบจริง ผลที่ได้มีดังนี้

ตารางที่ 4 ผลการทดลองยืนยันข้อมูลบนบอร์ด

ประเด็น	ข้อมูลเดิม	วิธีทดลองและผล	ผลต่อการออกแบบ
ทิศทาง UART ของ USB และ ESP8266	ลูกศรในคู่มือหน้า 6-7 ระบุว่า F2 และ A12 เป็นขาเข้า FPGA	ส่งข้อมูลทะลุ FPGA แล้วเปิด Serial Monitor เห็นข้อความบูตของ ESP ที่ COM3 และอัปโหลดโปรแกรม ESP ได้ (24 ต.ค. 2569) จึงยืนยันว่า G1 และ A10 เป็นขาเข้า	กำหนด usb_rx = G1, esp_tx = A10, usb_tx = F2, esp_rx = A12
ขาข้อมูลของ ADC	ป้ายชื่อในคู่มือสลับกัน	ยืนยันจากการทดลอง ADC ในรายวิชา ว่า G4 คือขาข้อมูลออกของ ADC	adc_dout = G4
ตารางขาหัว J5	ตารางในคู่มือเป็นเลขขาของรุ่น Spartan-6	เทียบกับไฟล์ XDC ทางการของบอร์ด	ใช้ขาตาม XDC ทางการ
LED บนบอร์ด	LED D2-D9 และ D10-D17 ใช้ขาร่วมกับ LCD และ 7-segment	ตรวจจากผังวงจร	ถอด J9 เสมอ, J4 ไม่จำเป็น
การขับพัดลม	ต่อพัดลมตรงกับขา FPGA	เว็บแสดงว่าเปิด แต่พัดลมไม่หมุน (24 ต.ค. 2569) เพราะขา LVCMOS33 จ่ายกระแสได้ราว 12 mA	ต้องมีวงจรขับ (ทรานซิสเตอร์ / MOSFET / โมดูล)
การอัปโหลดโปรแกรม ESP8266 ผ่านบอร์ด	คาดว่าอัปโหลดได้ตามปกติ	ขึ้นข้อความ Timed out waiting for packet header เพราะทาง FPGA มีแค่ TX/RX ไม่มี DTR/RTS	ต้องเข้า bootloader เอง (GPIO0 ลง GND + รีเซ็ตด้วย J11) และเพิ่มโหมดส่งข้อมูลทะลุด้วย SW8
Buzzer	คู่มือระบุว่าเป็น piezo	จ่าย 1 ค้างแล้วมีเสียง (24 ต.ค. 2569)	เสียงเตือนใช้ 1 ค้างเป็นจังหวะ ส่วนเพลงใช้คลื่นสี่เหลี่ยมตามความถี่โน้ต

3. ข้อกำหนดการออกแบบที่ได้จากการหาข้อมูล

1. สัญญาณนาฬิกาหลัก 50 MHz จากขา H11 ทุกวงจรเป็นแบบซิงโครนัสกับสัญญาณนาฬิกาเดียว
2. อินพุตจากภายนอกทุกเส้นต้องผ่านวงจรซิงโครไนซ์ 2 ชั้นก่อนใช้งาน เพราะมาไม่พร้อมกับสัญญาณนาฬิกา
3. UART ใช้ 115200 บิตต่อวินาที 8N1 ให้ตรงกับ ESP8266 และใช้ 50 MHz / 115200 ≈ 434 clock ต่อบิต
4. ADC ใช้ SCK ไม่เกิน 1 MHz และต้องเฉลี่ยค่า เพราะสัญญาณ LM35 เล็ก (ราว 12 ระดับ ADC ต่อ 1 °C)
5. LED ภายนอกต่อแบบ Active-HIGH ผ่านตัวต้านทาน 220 โอห์ม และขาเดียวต้องไม่จ่ายเกิน 12 mA
6. อุปกรณ์กำลังสูง (พัดลม) ต้องมีวงจรขับแยก
7. ต้องอัปโหลดโปรแกรม ESP8266 ผ่าน FPGA ได้ โดยไม่ต้องถอดสาย

4. ทฤษฎีที่เกี่ยวข้อง

4.1 Finite State Machine (FSM)

FSM คือรูปแบบการออกแบบวงจรดิจิทัลที่แบ่งการทำงานเป็นสถานะต่าง ๆ วงจรอยู่ได้ทีละหนึ่งสถานะ และเปลี่ยนสถานะตามเงื่อนไขของอินพุตทุกขอบขาขึ้นของสัญญาณนาฬิกา ทำให้ควบคุมลำดับการทำงานได้ชัดเจน โครงงานใช้ FSM กับโหมดไฟ การปรับแอร์ การรับ-ส่ง UART และ SPI การแยกเฟรมคำสั่ง ลำดับคำสั่งของจอ LCD และลำดับโน้ตของเพลง

4.2 ตัวนับและตัวแบ่งความถี่ (Counter / Clock Divider)

ตัวนับนับขอบสัญญาณนาฬิกา 50 MHz แล้วสร้างจังหวะที่ช้ากว่า เช่น นับครบ 434 ครั้งได้ 1 บิตของ UART นับครบ 100,000,000 ครั้งได้ 2 วินาที โครงงานใช้ตัวนับกับ baud rate, SCK ของ SPI, การสแกน 7-segment, จังหวะ buzzer, ความถี่โน้ตดนตรี และการหมดเวลาของเฟรม

4.3 การสื่อสารแบบ SPI

SPI เป็นการสื่อสารอนุกรมแบบซิงโครนัสที่มีสายนาฬิกา (SCK) สายข้อมูลเข้า สายข้อมูลออก และสายเลือกอุปกรณ์ (CS) ข้อมูลเลื่อนทีละบิตตามขอบนาฬิกา โครงงานใช้ SPI อ่าน ADC MCP3208 ซึ่งแปลงแรงดันจาก LDR และ LM35 เป็นค่าดิจิทัล 12 บิต

4.4 การสื่อสารแบบ UART

UART เป็นการสื่อสารอนุกรมแบบไม่มีสายนาฬิการ่วม ทั้งสองฝั่งต้องตั้งความเร็วเท่ากัน ข้อมูลหนึ่งไบต์ส่งเป็นเฟรมที่มี start bit (0) ข้อมูล 8 บิตเริ่มจากบิตต่ำสุด และ stop bit (1) ฝั่งรับตรวจจับขอบลงของ start bit แล้วอ่านแต่ละบิตตรงกลางช่วงบิต โครงงานใช้ UART 115200 บิตต่อวินาทีระหว่าง FPGA กับ ESP8266 และระหว่างบอร์ด 1 กับบอร์ด 2

4.5 การแสดงผล 7-segment แบบ Multiplex

7-segment 4 หลักใช้สาย segment ร่วมกัน จึงเปิดทีละหลักสลับกันเร็วพอที่ตามองเห็นว่าติดพร้อมกัน ก่อนแสดงต้องแปลงเลขฐานสองเป็นเลขฐานสิบทีละหลัก (BCD) แล้วถอดรหัสเป็นรูปแบบ segment

4.6 การกรองสัญญาณปุ่มกด (Debounce)

หน้าสัมผัสของปุ่มและสวิตช์จะเด้งหลายครั้งตอนกด วงจร debounce ยอมรับค่าใหม่เมื่อค่าคงที่ต่อเนื่องนานพอ แล้วใช้ตัวตรวจขอบขาขึ้น (edge detector) ทำให้การกดหนึ่งครั้งนับเป็นหนึ่งครั้งเท่านั้น

4.7 การควบคุมจอ LCD 16×2

จอ LCD ตระกูล HD44780 รับข้อมูล 8 บิต ขา RS เลือกว่าเป็นคำสั่งหรือตัวอักษร และขา EN เป็นจังหวะให้จออ่านข้อมูล ก่อนใช้งานต้องส่งชุดคำสั่งเริ่มต้นตามลำดับและรอเวลาที่จอกำหนด จึงต้องใช้ FSM ร่วมกับตัวนับเวลา

4.8 ตัวถอดรหัสและวงจรบวกลบ (Decoder / Adder)

ตัวถอดรหัสแปลงรหัสไม่กี่บิตเป็นเอาต์พุตที่ทำงานเพียงเส้นเดียว ใช้เลือกไฟโซน 1 จาก 3 ดวงและไฟชั้น 1 จาก 9 ดวง ส่วนวงจรบวกลบใช้คำนวณจำนวนที่นั่งว่างและปรับอุณหภูมิแอร์ทีละ 0.5 °C

อุปกรณ์และเครื่องมือ

1. อุปกรณ์

1.	บอร์ด FPGA EDGE Spartan 7 (XC7S15-FTGB196)	2 บอร์ด
2.	โมดูลเซนเซอร์ควัน MQ-2	1 ตัว
3.	พัดลม DC ขนาดเล็ก พร้อมวงจรขับ	1 ชุด
4.	LED สีขาว (ไฟห้อง)	3 ดวง
5.	LED ไฟโซน สีแดง น้ำเงิน เขียว	3 ดวง
6.	LED ไฟชั้น สีแดง น้ำเงิน เหลือง (เหลืองใช้แทนชั้นสีเขียว)	9 ดวง
7.	ตัวต้านทาน 220 โอห์ม	15 ตัว
8.	คอมพิวเตอร์ของห้องสมุด (เปิดเว็บค้นหาหนังสือ)	1 เครื่อง
9.	โมเดลห้องสมุดจำลอง	1 ชุด

2. เครื่องมือและซอฟต์แวร์

ตารางที่ 5 เครื่องมือในการพัฒนา

เครื่องมือ	ใช้ทำอะไร
Vivado 2025.2 (โหมด batch)	สังเคราะห์ วางวงจร และสร้างไฟล์ .bit สำหรับ xc7s15ftgb196-1
openFPGALoader	โหลดไฟล์ลง FPGA ชั่วคราว (SRAM) หรือเขียนลง Flash ถาวรผ่าน JTAG
Arduino IDE 2 + ESP8266 Core 3.1.2	เขียนและอัปโหลดโปรแกรม ESP8266 (บอร์ด Generic ESP8266, Reset Method no dtr)
Google Chrome	ทดสอบหน้าเว็บและการแสดงผลที่ความกว้างต่าง ๆ
Git	เก็บประวัติการแก้ไขโค้ดและเอกสารทั้งหมด

ภาษาที่ใช้: VHDL สำหรับวงจรบน FPGA และ C++ (Arduino) กับ HTML/JavaScript สำหรับ ESP8266

อุปกรณ์บนบอร์ด FPGA ที่ใช้ ได้แก่ สวิตช์เลื่อน ปุ่มกด 3 ปุ่ม LDR และ LM35 ผ่าน ADC MCP3208, 7-segment 4 หลัก, จอ LCD 16×2, buzzer, โมดูล Wi-Fi ESP8266 และ USB-UART

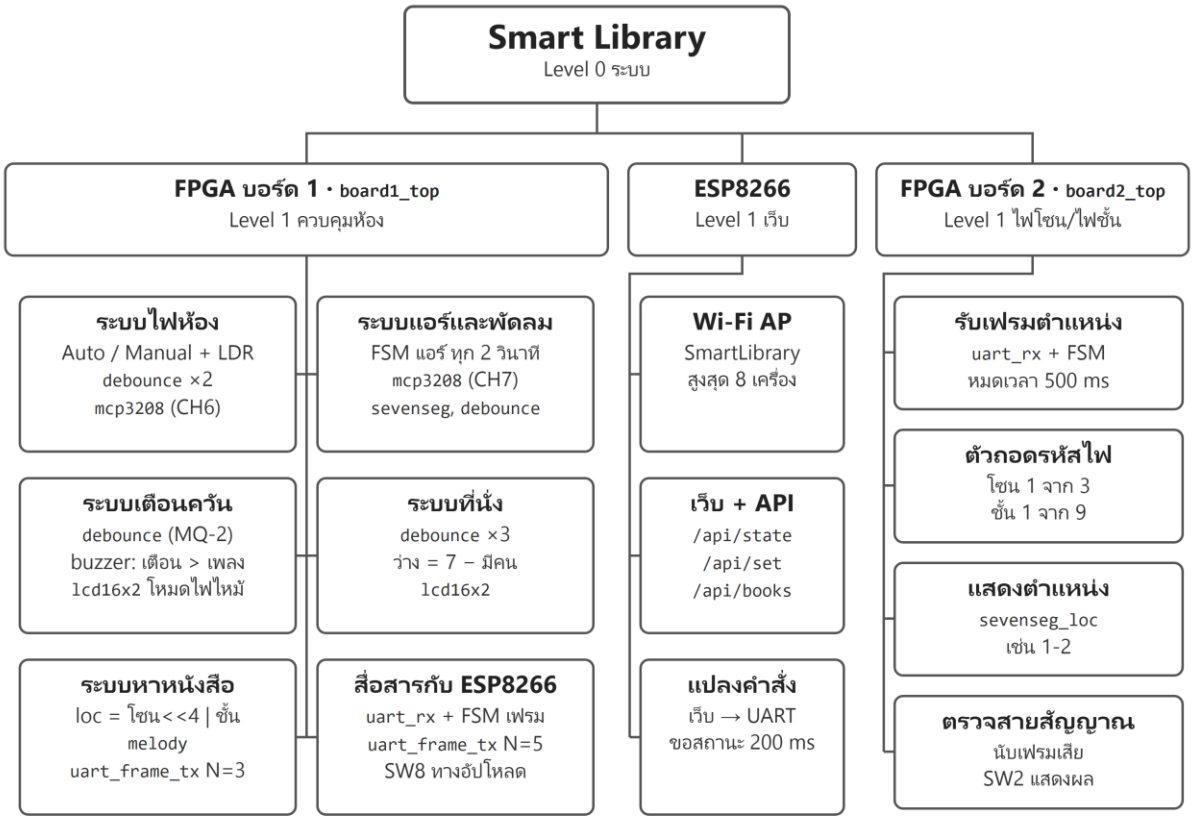
การออกแบบและการพัฒนา

1. แนวคิดการออกแบบแบบ Top-Down

การออกแบบส่วน FPGA เริ่มจากความต้องการของทั้งระบบ แล้วแตกออกเป็นส่วนย่อยทีละระดับ แต่ละระดับกำหนดหน้าที่และสัญญาณเชื่อมต่อ (interface) ให้ชัดก่อนลงรายละเอียดภายใน เมื่อแตกถึงระดับที่เขียนเป็น entity ของ VHDL ได้โดยตรง จึงลงมือเขียนและทดสอบทีละ entity แล้วประกอบกลับขึ้นไปทีละระดับ

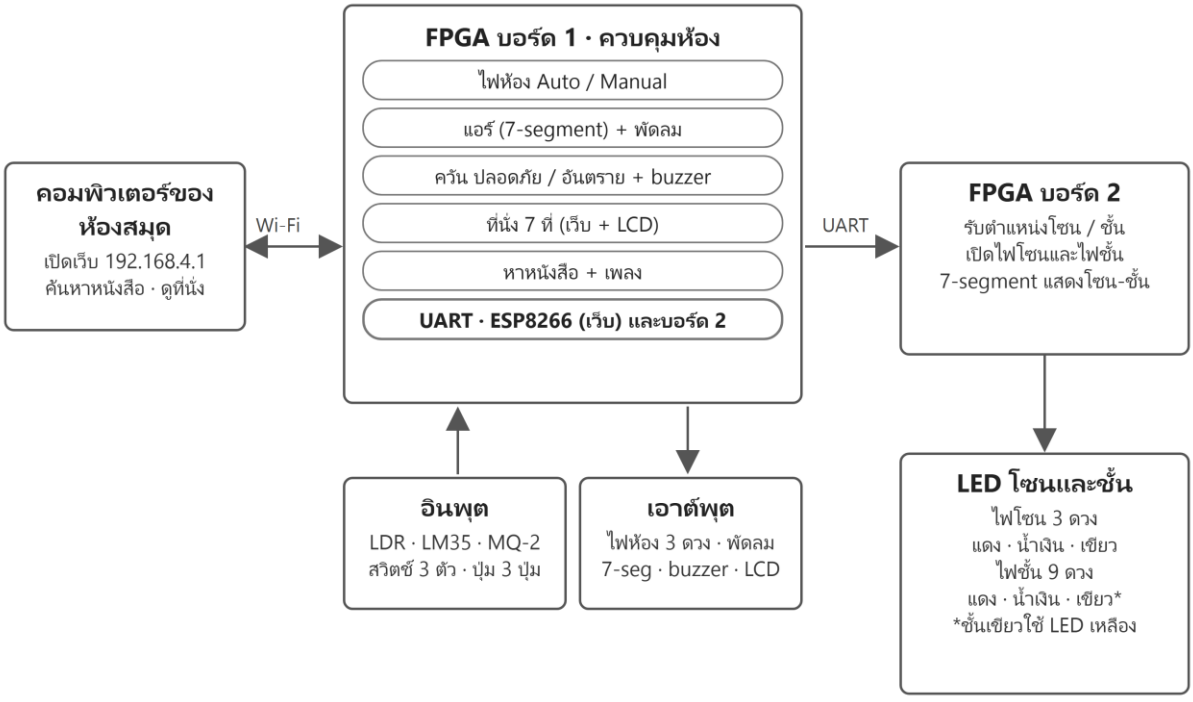
ตารางที่ 6 ระดับของการออกแบบ Top-Down

ระดับ	สิ่งที่ออกแบบ	ผลลัพธ์
Level 0 ระบบ	ความต้องการทั้งหมด และการแบ่งงานระหว่าง FPGA กับ ESP8266	แผนภาพระบบ (ภาพที่ 2)
Level 1 บอร์ด	หน้าที่ของแต่ละบอร์ด ขาที่ใช้ และโปรโตคอลระหว่างบอร์ด	top entity: board1_top, board2_top
Level 2 ระบบย่อย	ระบบไฟ แอร์และพัดลม ควัน ที่นั่ง หาหนังสือ การสื่อสาร	แผนภาพบล็อก (ภาพที่ 3 และ 4)
Level 3 โมดูล	entity ที่นำกลับมาใช้ซ้ำได้ และ FSM ภายใน	uart_rx, uart_frame_tx, debounce, mcp3208, sevenseg, lcd16x2, melody, sevenseg_loc



ภาพที่ 1 ลำดับชั้นการออกแบบ Top-Down ของ Smart Library

2. Level 0: ระดับระบบ



ภาพที่ 2 แผนภาพการทำงานของระบบ Smart Library

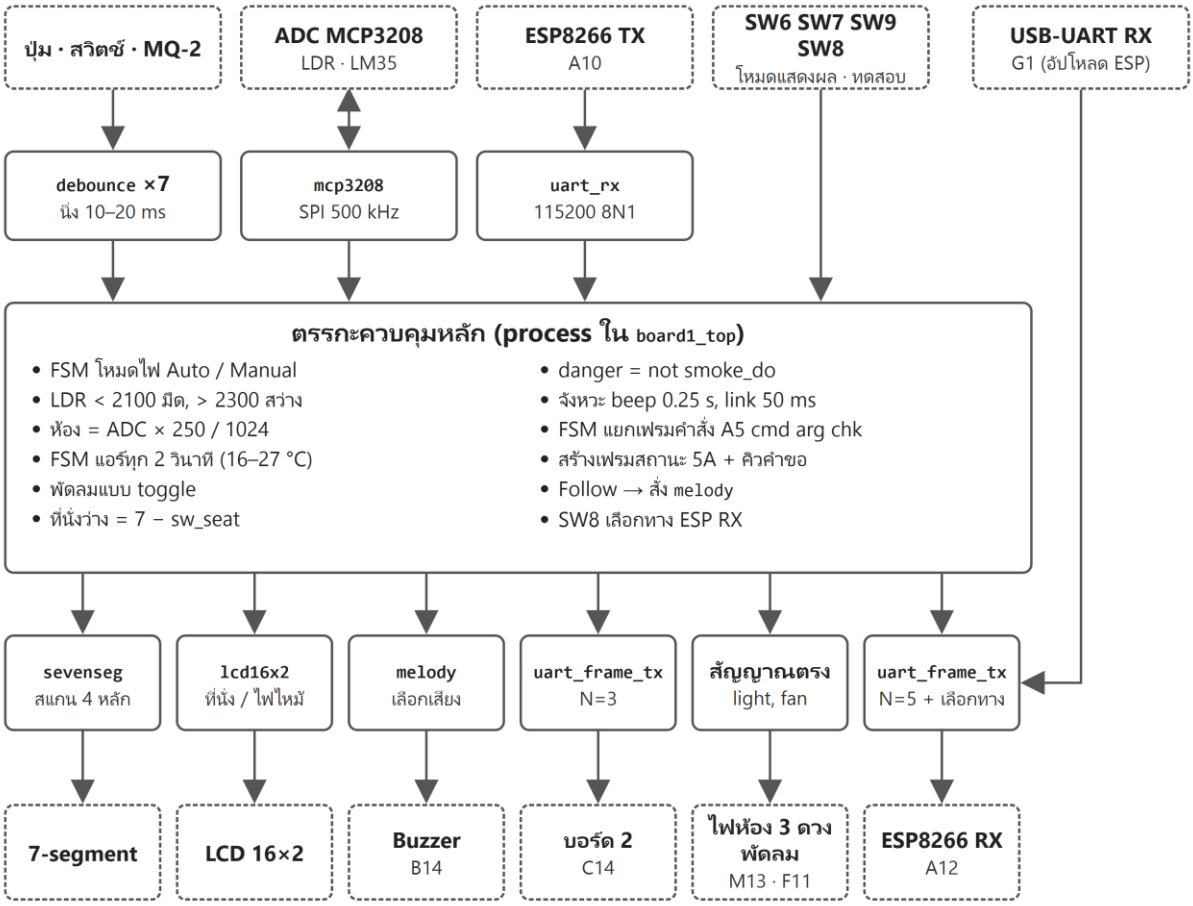
ระดับระบบแบ่งงานเป็น 3 ส่วน คือ FPGA บอร์ด 1 FPGA บอร์ด 2 และ ESP8266 เหตุผลของการแบ่งมีดังนี้

- **แยกบอร์ด 2 สำหรับไฟชั้นหนังสือ:** บอร์ด 1 ใช้ขา I/O ไป 46 ขาจาก 100 ขา ไฟโซนและไฟชั้นอีก 12 ดวงอยู่คนละมุมของโมเดล การแยกบอร์ดทำให้เดินสายสั้นลง และบอร์ด 2 ต้องการสัญญาณจากบอร์ด 1 เพียงเส้นเดียว
- **UART สายเดียวระหว่างบอร์ด:** ข้อมูลไหลทางเดียว (บอร์ด 1 ⟶ บอร์ด 2) ใช้สายสัญญาณ 1 เส้นกับ GND ร่วม และใช้ตัวรับ uart_rx ตัวเดียวกับที่ใช้รับจาก ESP8266
- **ESP8266 บนบอร์ดเป็นเว็บเซิร์ฟเวอร์:** มีอยู่บนบอร์ดแล้ว ไม่ต้องเพิ่มฮาร์ดแวร์ ให้ Wi-Fi และหน้าเว็บได้ทันที ส่วน FPGA เหมาะกับงานควบคุมแบบขนานมากกว่างานเว็บ
- **ใช้คอมพิวเตอร์ของห้องสมุด:** ผู้ใช้ไม่ต้องเชื่อมต่อ Wi-Fi เอง และจำนวนเครื่องที่เชื่อมต่อคงที่ จึงไม่เกินขีดจำกัดของ ESP8266

3. Level 1: ระดับบอร์ด

3.1 บอร์ด 1 (board1_top)

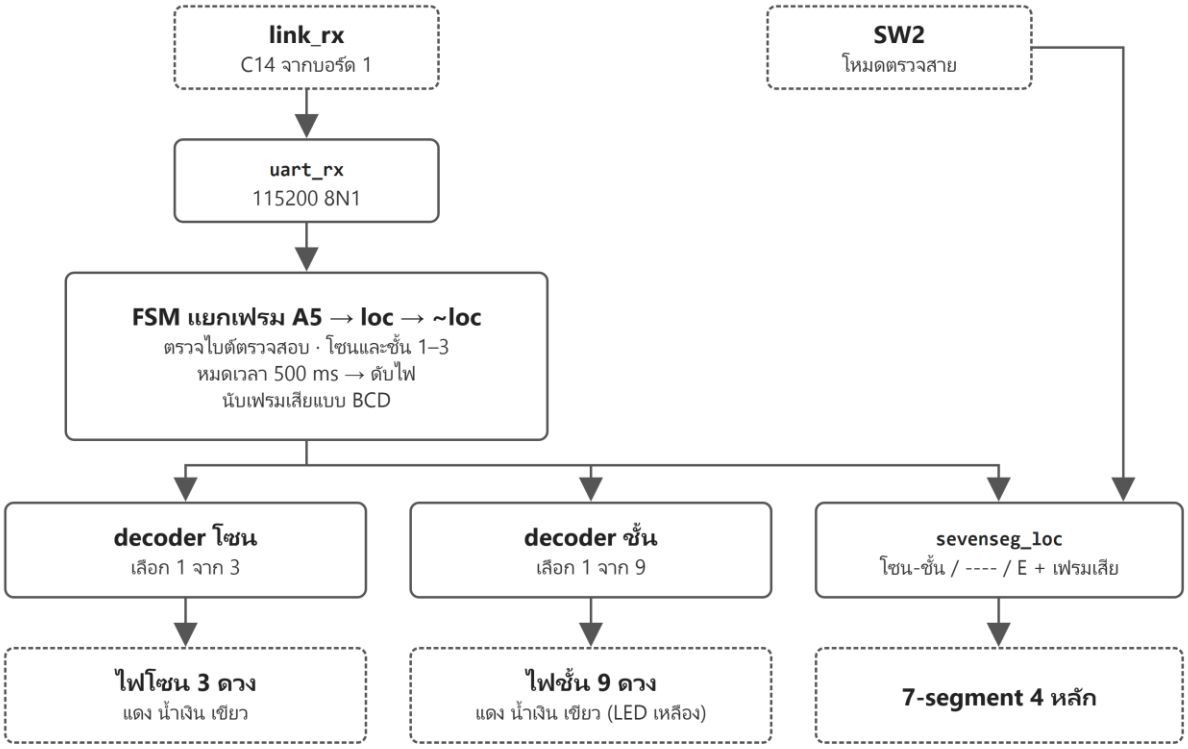
บอร์ด 1 รับอินพุตจากปุ่ม สวิตช์ เซนเซอร์ควัน และ ADC แล้วส่งเข้าตรรกะควบคุมหลักใน board1_top ซึ่งเป็น process เดียวที่ทำงานทุกขอบขาขึ้นของสัญญาณนาฬิกา ผลลัพธ์ส่งไปยังโมดูลแสดงผลและโมดูลส่งข้อมูล ดังภาพที่ 3



ภาพที่ 3 แผนภาพบล็อกภายใน board1_top

3.2 บอร์ด 2 (board2_top)

บอร์ด 2 รับเฟรมตำแหน่งจากบอร์ด 1 ตรวจความถูกต้อง แล้วใช้ decoder เลือกไฟโซน 1 ดวงจาก 3 ดวง และไฟชั้น 1 ดวงจาก 9 ดวง พร้อมแสดงตำแหน่งบน 7-segment ถ้าไม่ได้รับเฟรมที่ถูกต้องเกิน 500 ms จะดับไฟทั้งหมด เพื่อไม่ให้ไฟค้างเมื่อสายหลุด



ภาพที่ 4 แผนภาพบล็อกภายใน board2_top

3.3 การกำหนดขา

ขาทั้งหมดใช้มาตรฐาน LVCMOS33 ขาที่สำคัญสรุปไว้ในตารางที่ 7 ตารางขาฉบับเต็มอยู่ในภาคผนวก ก

ตารางที่ 7 ขาที่สำคัญ

สัญญาณ	บอร์ด	ขา	หมายเหตุ
clk	1, 2	H11	ออสซิลเลเตอร์ 50 MHz
esp_tx / esp_rx	1	A10 / A12	UART กับ ESP8266 (ยืนยันทิศบนบอร์ด)
usb_rx / usb_tx	1	G1 / F2	UART กับคอมพิวเตอร์ ใช้ตอนอัปโหลด ESP
link_tx ⟶ link_rx	1 ⟶ 2	C14 ⟶ C14	สายระหว่างบอร์ด + GND ร่วม
room_led	1	M13	LED ขาว 3 ดวงต่อขนาน
fan	1	F11	สัญญาณไปวงจรขับพัดลม
smoke_do	1	L14	MQ-2 DO
zone_led[2:0]	2	L14 M13 H14	ไฟโซน แดง น้ำเงิน เขียว
shelf_led[8:0]	2	J5 9 ขา	ไฟชั้น 9 ดวง

4. Level 2–3: ระดับโมดูล

หัวข้อนี้อธิบาย entity ทุกตัว ทั้งหน้าที่ หลักการ ค่าที่เลือก และเหตุผลของการเลือก

4.1 uart_rx (ตัวรับ UART)

รับข้อมูลอนุกรม 8N1 ใช้ทั้งบนบอร์ด 1 (รับจาก ESP8266) และบอร์ด 2 (รับจากบอร์ด 1) ทำงานเป็น FSM 4 สถานะดังภาพที่ 5 (ก)

- ซิงโครไนซ์ขา rx ด้วย flip-flop 2 ชั้นก่อนใช้ ลดปัญหา metastability
- เมื่อเห็นขอบลงของ start bit จะรอครึ่งบิต (217 clock) แล้วตรวจซ้ำว่ายังเป็น 0 ถ้าไม่ใช่ถือเป็นสัญญาณรบกวนและกลับไป IDLE
- จากนั้นเก็บข้อมูลทุก 434 clock ซึ่งตรงกลางของแต่ละบิตพอดี ความเร็วจริงคือ 50 MHz / 434 = 115,207 บิตต่อวินาที คลาดจาก 115,200 เพียง 0.006%
- ส่ง valid เพียง 1 clock เมื่อ stop bit เป็น 1 เท่านั้น ไบต์ที่ stop bit ผิดจะถูกทิ้ง

4.2 uart_frame_tx (ตัวส่งเฟรม UART)

ส่งข้อมูลหลายไบต์ต่อเนื่องเป็นเฟรมเดียว กำหนดจำนวนไบต์ด้วย generic N บอร์ด 1 ใช้ 2 ตัว คือ N = 5 สำหรับเฟรมสถานะไป ESP8266 และ N = 3 สำหรับเฟรมตำแหน่งไปบอร์ด 2

- เมื่อได้ start จะโหลดทุกไบต์พร้อม start bit และ stop bit ลงชิฟต์รีจิสเตอร์ขนาด 10 × N บิต แล้วเลื่อนออกทีละบิตทุก 434 clock
- เลือกส่งทั้งเฟรมในครั้งเดียว เพื่อให้ไบต์ในเฟรมเดียวกันมาจากค่าชุดเดียวกันเสมอ ไม่มีไบต์จากค่าเก่าปนค่าใหม่
- สัญญาณ busy ใช้บอกตรรกะควบคุมว่ายังส่งไม่เสร็จ

4.3 debounce (กรองสัญญาณปุ่มและสวิตช์)

ซิงโครไนซ์อินพุต 2 ชั้น แล้วนับว่าค่าใหม่คงที่ต่อเนื่องครบเวลาที่กำหนดหรือไม่ ถ้าค่าเปลี่ยนกลับก่อนครบจะเริ่มนับใหม่ ปุ่มกดใช้ 20 ms (1,000,000 clock) เพราะหน้าสัมผัสปุ่มเด้งนานกว่า ส่วนสวิตช์เลื่อนและสัญญาณควันใช้ 10 ms ตัวกรองของ MQ-2 ตั้งค่าเริ่มต้นเป็น 1 (ไม่มีควัน) เพื่อไม่ให้เตือนผิดตอนเปิดเครื่อง บอร์ด 1 ใช้ทั้งหมด 7 ตัว

4.4 mcp3208 (อ่าน ADC ผ่าน SPI)

- สร้าง SCK 500 kHz จากการนับ 50 clock ต่อครึ่งคาบ ต่ำกว่าขีดจำกัดของชิปที่ 3.3 V ครึ่งหนึ่ง จึงมีระยะเผื่อ
- ส่งบิต start, SGL = 1 (single-ended) และหมายเลขช่อง D2-D0 แล้วอ่านผล 12 บิต สลับอ่าน CH6 (LDR) และ CH7 (LM35)
- เว้นช่วง 200 µs ระหว่างการแปลงแต่ละครั้ง ให้ตัวเก็บประจุภายใน ADC ชาร์จเต็ม
- เฉลี่ยค่าครั้งละ 1,024 ตัวอย่างต่อช่อง ด้วยการบวกสะสมแล้วเลื่อนบิตขวา 10 บิต (หารด้วย 1,024 โดยไม่ต้องใช้วงจรหาร) ค่าใหม่ออกทุกราว 0.5 วินาที
- เหตุผลของการเฉลี่ย: LM35 ให้ 10 mV/°C และ ADC มีความละเอียด 3.3 V / 4096 ≈ 0.8 mV จึงได้ราว 12 ระดับต่อ 1 °C สัญญาณรบกวนเพียงไม่กี่ mV ทำให้ค่ากระโดดได้มาก

4.5 sevenseg (แสดงผล 7-segment บอร์ด 1)

- สแกนทีละหลัก หลักละ 1 ms (50,000 clock) ครบ 4 หลักใน 4 ms หรือ 250 ครั้งต่อวินาที เร็วพอที่ตาไม่เห็นการกะพริบ
- แปลงเลขฐานสองเป็น BCD ด้วยอัลกอริทึม double dabble (เลื่อนบิตแล้วบวก 3 ให้หลักที่มากกว่า 4) เป็นวงจรเชิงผสมล้วน
- อุณหภูมิเก็บในหน่วย 0.5 °C จึงคูณ 5 ให้เป็นหน่วย 0.1 °C ก่อนแปลง เช่น 49 × 5 = 245 แสดงเป็น 24.5C โดยเปิดจุดทศนิยมที่หลักที่ 2
- SW7-SW6 เลือกแสดงอุณหภูมิแอร์ ค่าดิบ LDR หรืออุณหภูมิห้อง ใช้ปรับเทียบเกณฑ์และสูตรบนบอร์ดจริง

4.6 lcd16x2 (จอ LCD)

- รอ 50 ms หลังเปิดเครื่อง แล้วส่งคำสั่ง 0x38 (8 บิต 2 บรรทัด) 3 ครั้ง, 0x0C (เปิดจอ), 0x01 (ล้างจอ), 0x06 (เลื่อนตำแหน่งอัตโนมัติ) โดยรอคำสั่งละ 5 ms
- หลังจากนั้นเขียนตัวอักษรทั้ง 32 ตัวซ้ำวนไปเรื่อย ๆ ตัวละ 50 µs ครบหนึ่งรอบใน 1.7 ms ค่าที่เปลี่ยนจึงขึ้นจอทันทีโดยไม่ต้องมีวงจรตรวจการเปลี่ยนแปลง
- ปกติแสดง Free seats และ Used seats โดยแปลงจำนวนเป็น ASCII ด้วยการบวก 48
- เมื่อควันอันตรายแสดง !! FIRE ALARM !! และบรรทัด 2 EVACUATE NOW กะพริบตามจังหวะ buzzer จอไม่มีอักษรไทยจึงใช้ภาษาอังกฤษ

4.7 melody (เพลงตอนกด Follow)

เก็บเพลงเป็นตารางคงที่ แต่ละโน้ตมีค่าครึ่งคาบ (จำนวน clock) และความยาว (ms) ค่าครึ่งคาบคำนวณจาก 50 MHz / (2 × ความถี่) เช่น E6 = 1318.5 Hz ได้ 18,961 clock ตัวนับครึ่งคาบสร้างคลื่นสี่เหลี่ยม ตัวนับ ms นับความยาวโน้ต และเงียบช่วงท้ายโน้ต 25 ms ให้โน้ตซ้ำแยกกันได้ยิน เพลงที่ใช้คือท่อนเปิด Super Mario Bros. ยาวประมาณ 2 วินาที

```
constant SONG : song_t := (
  (E6, 125), (E6, 125), (R, 125), (E6, 125),
  (R, 125), (C6, 125), (E6, 250), (G6, 250),
  (R, 250), (G5, 250)
);  -- (ครึ่งคาบเป็น clock, ความยาว ms) / R = พัก
```

4.8 ตรรกะควบคุมหลักใน board1_top

ตารางที่ 8 ส่วนประกอบของตรรกะควบคุมหลัก

ส่วน	หลักการ	เหตุผลของการออกแบบ
ไฟห้อง	FSM 2 สถานะ AUTO / MANUAL, Auto: LDR < 2100 = มืด, > 2300 = สว่าง	ใช้ hysteresis 2 ค่าเพื่อไม่ให้ไฟกะพริบตอนแสงก้ำกึ่ง และตอนเข้า Manual ให้ไฟคงสถานะเดิม
อุณหภูมิห้อง	ห้อง (0.5 °C) = ADC × TEMP_SCALE / 1024 ด้วยตัวคูณ DSP 1 ตัว แล้วเลื่อนบิต 10 บิต	หลีกเลี่ยงวงจรหาร ตามทฤษฎีคือ 165 ค่าปัจจุบัน 250 ต้องปรับเทียบกับเทอร์โมมิเตอร์
แอร์	FSM HOLD / COOL / WARM ประเมินทุก 2 วินาที ห้อง > 26 °C ลด 0.5 °C, < 24 °C เพิ่ม 0.5 °C จำกัด 16–27 °C	ช่วง 24–26 °C ไม่ปรับ ป้องกันการแกว่ง และปรับทีละน้อยให้เห็นการเปลี่ยนบนโมเดล
พัดลม	ตรวจขอบขาขึ้นของปุ่ม แล้วสลับสถานะ (toggle flip-flop)	กดค้างก็สลับเพียงครั้งเดียว
ที่นั่ง	used = sw_seat (ฐานสอง 3 บิต), free = 7 − used	สวิตช์ 3 ตัวแทนได้ 0–7 ที่นั่ง แทนแบบ 1 สวิตช์ต่อ 1 ที่ซึ่งได้แค่ 3 ที่
เตือนควัน	danger = not smoke_do, buzzer = alarm ? beep : เพลง	เสียงเตือนมีความสำคัญสูงสุดเสมอ
คำสั่งจาก ESP	FSM แยกเฟรม A5 cmd arg chk (ภาพที่ 5 ข)	ตรวจ checksum ก่อนทำคำสั่ง และตอบสถานะทุกคำสั่งที่ถูกต้อง

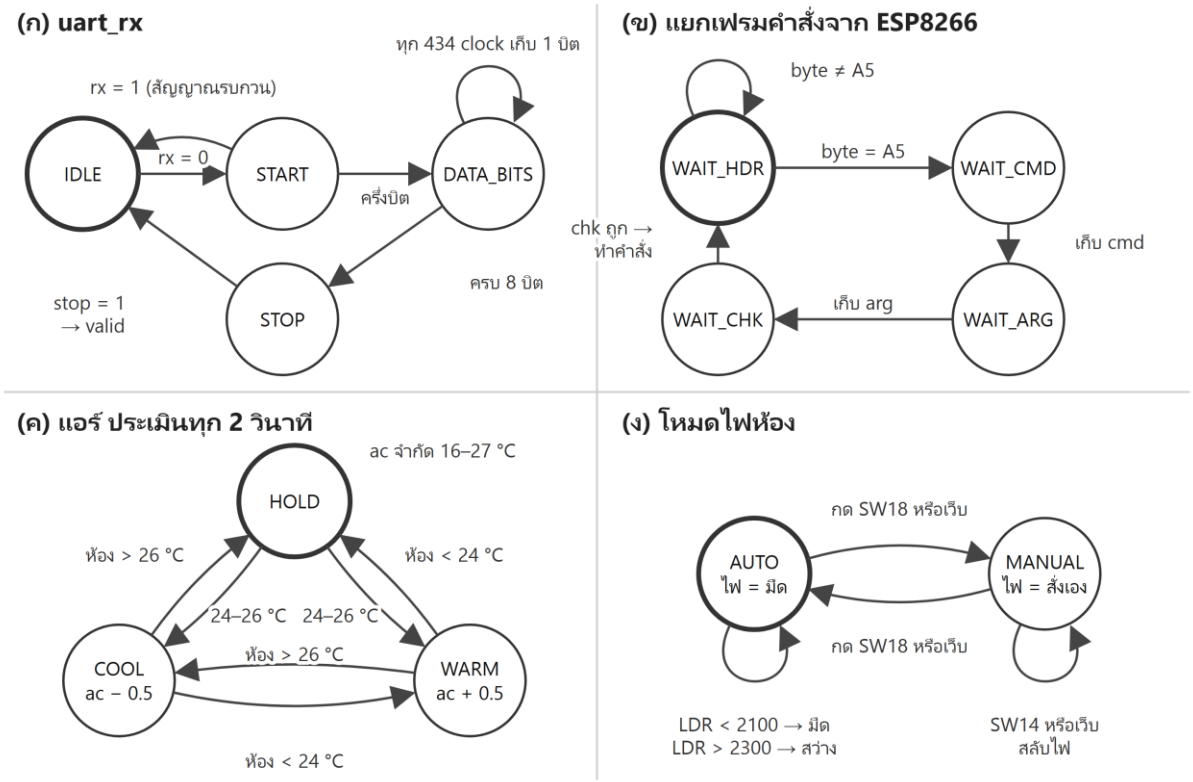
ส่วน	หลักการ	เหตุผลของการออกแบบ
ส่งสถานะ	ตั้งธงคำขอ status_req แล้วส่งเมื่อตัวส่งว่าง	คำขอที่มาตอนกำลังส่งเฟรมก่อนหน้าจะไม่หาย
ส่งตำแหน่ง	ส่งเฟรมตำแหน่งให้บอร์ด 2 ทุก 50 ms	ถ้าเฟรมหายจะได้เฟรมใหม่ภายใน 50 ms บอร์ด 2 ไม่ต้องส่งตอบกลับ
อัปโหลด ESP	esp_rx = sw_prog ? usb_rx : fpga_tx และ usb_tx = esp_tx	อัปโหลดโปรแกรม ESP ผ่าน USB ของบอร์ดได้โดยไม่ต้องเปลี่ยนสาย

```
when x"02" =>                    -- หาหนังสือ
  loc <= arg;                    -- ตำแหน่ง = โซน << 4 | ชั้น
  if arg /= x"00" then           -- Follow (ไม่ใช่ยกเลิก)
    song_start <= '1';           -- สั่ง melody เริ่มเล่น
  end if;
```

4.9 บอร์ด 2: แยกเฟรม decoder และ sevenseg_loc

- FSM แยกเฟรม WAIT_HDR ⟶ WAIT_LOC ⟶ WAIT_CHK ยอมรับเฟรมเมื่อไบต์ตรวจสอบเท่ากับ ~loc ถ้าไบต์ตรวจสอบผิดแต่เป็น A5 จะถือว่าเป็นหัวเฟรมใหม่ทันที จึงกลับเข้าจังหวะได้เร็ว
- decoder โซนและ decoder ชั้นเขียนด้วยคำสั่ง with-select ค่านอกช่วงโซน 1–3 ชั้น 1–3 จะดับไฟทั้งหมด
- ตัวนับเฟรมเสียเป็น BCD 3 หลักโดยตรง (000–999 หยุดที่ 999) จึงนำไปแสดงได้ทันทีโดยไม่ต้องแปลง
- sevenseg_loc แสดง โซน-ชั้น เช่น 1-2, แสดง ---- เมื่อไม่มีตำแหน่ง และจุดหลักขวาสุดสลับทุกเฟรมที่ถูกต้อง ใช้เป็นสัญญาณชีพของสาย

5. แผนภาพสถานะ (FSM)



ภาพที่ 5 แผนภาพสถานะของ FSM หลักในระบบ

6. การออกแบบโปรโตคอลสื่อสาร

ทุกเส้นใช้ UART 115200 8N1 ทุกเฟรมเริ่มด้วยไบต์หัวเฟรมและปิดท้ายด้วยไบต์ตรวจสอบ ซึ่งเป็นค่ากลับบิตของ XOR ของข้อมูลทั้งหมด การกลับบิตทำให้เฟรมที่เป็นศูนย์ทั้งหมด (เช่นสายลอยหรือขาดเป็น 0) ไม่ผ่านการตรวจ

ตารางที่ 9 รูปแบบเฟรม

ทิศทาง	เฟรม	ความถี่
ESP8266 ⟶ บอร์ด 1	A5 cmd arg ~(cmd ⊕ arg)	เมื่อมีคำสั่ง และขอสถานะทุก 200 ms
บอร์ด 1 ⟶ ESP8266	5A flags ac loc ~(flags ⊕ ac ⊕ loc)	ตอบทุกคำสั่งที่ถูกต้อง
บอร์ด 1 ⟶ บอร์ด 2	A5 loc ~loc	ทุก 50 ms

ตารางที่ 10 คำสั่งจาก ESP8266

cmd	ความหมาย	arg
00	ขอสถานะ	00
01	โหมดไฟ	1 = Manual, 0 = Auto
02	หาหนังสือ	โซน << 4 | ชั้น, 00 = ยกเลิก
03	ไฟห้อง (Manual เท่านั้น)	1 = ติด
04	พัดลม	1 = เปิด

ตารางที่ 11 ความหมายของไบต์ flags (ac = อุณหภูมิแอร์หน่วย 0.5 °C)

บิตของ flags	ความหมาย
0	ไฟห้องติด
1	โหมด Manual
2	พัดลมเปิด
3	ควันอันตราย
4–6	จำนวนที่นั่งที่มีคน 0–7
7	0 (สำรอง)

7. การออกแบบส่วน ESP8266 และเว็บ

- ESP8266 เปิด Wi-Fi แบบ Access Point ชื่อ SmartLibrary (รหัส library123) ที่ 192.168.4.1 รองรับสูงสุด 8 เครื่อง
- API: /api/state ส่งสถานะเป็น JSON, /api/set?lm=&lo=&fan=&fol= ส่งคำสั่ง, /api/books ส่งรายการหนังสือ 9 เล่มพร้อมโซน ชั้น และจำนวนคงเหลือ
- หน้าเว็บขอสถานะทุก 1.5 วินาที (polling) เพราะ ESP8266WebServer รองรับการเชื่อมต่อแบบขอ-ตอบได้ง่ายที่สุด และ 1.5 วินาทีเร็วพอสำหรับการแจ้งเตือนควัน
- หน้า Seat แสดงแถบที่นั่ง 1–7 เติมช่องที่มีคนจากซ้ายไปขวา หน้า Search แสดงหนังสือทุกเล่มแล้วกรองตามคำค้นหาทันที

- เมื่อควันอันตราย เว็บแสดงหน้าต่างเตือนเต็มจอทุกหน้าพร้อมเสียงไซเรนแบบ Yelp (700⟶1500 Hz 4 รอบต่อวินาที) ซึ่งสร้างด้วย Web Audio API และไม่ต้องเก็บไฟล์เสียงในหน่วยความจำของ ESP8266

8. การออกแบบวงจรภายนอก

ตารางที่ 12 วงจรภายนอกบอร์ด

วงจร	การต่อ	การคำนวณและเหตุผล
LED ไฟโซนและไฟชั้น	ขา FPGA ⟶ R 220 โอห์ม ⟶ LED ⟶ GND	LED แดง Vf ≈ 2.0 V: (3.3 – 2.0) / 220 ≈ 5.9 mA ไม่เกิน 12 mA ต่อขา
ไฟห้อง 3 ดวง	ขา M13 ต่อ (R 220 โอห์ม + LED) 3 ชุดขนานกันลง GND	LED ขาว Vf ≈ 3.0 V ได้ราว 1.4 mA ต่อดวง รวมราว 4 mA แต่ละดวงต้องมีตัวต้านทานของตัวเอง ความสว่างจึงเท่ากัน
ชั้นสีเขียว	ใช้ LED สีเหลืองแทน LED สีเขียว	LED เขียวแบบสว่าง (InGaN) มี Vf ราว 3 V ที่ไฟ 3.3 V จึงได้กระแสน้อยและหรี่ LED เหลือง Vf ≈ 2.1 V ได้ราว 5.5 mA สว่างกว่าชัดเจน ส่วนไฟโซนยังใช้สีเขียวตามสีโซน
พัดลม	F11 ⟶ R 1 กิโลโอห์ม ⟶ ทรานซิสเตอร์ NPN (หรือ MOSFET logic-level) ⟶ พัดลม + ไดโอดคร่อม	ขา FPGA จ่ายกระแสไม่พอสำหรับมอเตอร์ ไดโอดป้องกันแรงดันย้อนตอนตัดไฟมอเตอร์
MQ-2	DO ⟶ L14 (ระดับ 3.3 V)	ใช้ขาดิจิทัล ไม่ต้องใช้ ADC ปรับความไวที่ตัวต้านทานปรับค่าบนโมดูล
สายระหว่างบอร์ด	C14 ⟶ C14 และ GND ⟶ GND	ต้องต่อ GND ร่วม มิฉะนั้นระดับสัญญาณไม่มีจุดอ้างอิงและเฟรมเสีย

9. ลำดับการพัฒนา

ออกแบบจากบนลงล่าง (Top-Down) แต่ลงมือเขียนและทดสอบจากล่างขึ้นบน (Bottom-Up) เพื่อให้แต่ละ entity ผ่านการทดสอบก่อนนำไปประกอบ

1. uart_rx และ uart_frame_tx: ทดสอบด้วยการส่งข้อมูลทะลุ FPGA ระหว่างคอมพิวเตอร์กับ ESP8266
2. mcp3208 และ sevenseg: แสดงค่าดิบ LDR และอุณหภูมิบน 7-segment
3. debounce, lcd16x2 และตรรกะที่นั่ง: ตรวจตัวเลขบน LCD เทียบกับสวิตช์
4. ตรรกะไฟ แอร์ พัดลม และเตือนควัน ใน board1_top
5. โปรแกรม ESP8266 และหน้าเว็บ เชื่อมกับบอร์ด 1 ผ่านโปรโตคอลในหัวข้อที่ 6
6. บอร์ด 2 และสายระหว่างบอร์ด พร้อมโหมดตรวจสาย
7. รวมระบบ ประกอบโมเดล และเพิ่มความสามารถ (ปุ่มสลับโหมด, ที่นั่ง 7 ที่, เพลง, ข้อความไฟไหม้บน LCD)

10. ขั้นตอนการ build และอัปโหลด

```
# สร้างไฟล์ .bit (รันในโฟลเดอร์ FPGA_Builder_Package/build)
vivado -mode batch -source build_board1.tcl -nojournal -log
vivado_board1.log
vivado -mode batch -source build_board2.tcl -nojournal -log
vivado_board2.log


# โหลดลง FPGA
upload_board1_sram.bat      (ชั่วคราว หายเมื่อไฟดับหรือกด SW1)
upload_board1_prom.bat      (เขียน Flash ถาวร แล้วกด SW1)
```

การอัปโหลดโปรแกรม ESP8266: ตั้ง SW8 = 1 ต่อ GPIO0 ที่หัว J10 ลง GND ย้าย J11 Disable ⟶ Enable เพื่อรีเซ็ต ESP เข้า bootloader แล้วกด Upload ใน Arduino IDE เสร็จแล้วถอด GPIO0 รีเซ็ต ESP อีกครั้ง และตั้ง SW8 = 0 อีกทางหนึ่งคือโหลด esp_upload_top.bit ซึ่งเป็นวงจรส่งข้อมูลทะลุอย่างเดียว

11. ปัญหาที่พบและการแก้ไข

ตารางที่ 13 ปัญหาที่พบระหว่างพัฒนาและวิธีแก้

ปัญหา	สาเหตุ	การแก้ไข
ESP8266 ไม่ตอบสนองตอนต่อสายตามคู่มือ	ลูกศร UART ในคู่มือกลับด้าน	ทดลองยืนยันทิศ แล้วกำหนดขาใหม่
อัปโหลด ESP ไม่ได้ (Timed out waiting for packet header)	ไม่มีสาย DTR/RTS ผ่าน FPGA	เข้า bootloader เอง, Reset Method = no dtr, เพิ่ม SW8 และ esp_upload.vhd
พัดลมไม่หมุน แม้สถานะเป็น On	ขา FPGA จ่ายกระแสไม่พอ	เพิ่มวงจรขับพัดลม
บอร์ด 2 สลับ 1-1 กับ ---- ไปมา	เฟรมเสียจากสาย GND ไม่ร่วมหรือหลวม	ต่อ GND ร่วม และเพิ่มตัวนับเฟรมเสีย + จุดกะพริบ ไว้ตรวจสาย
เว็บไม่มีเสียงเตือนบนบางเครื่อง	เบราว์เซอร์ไม่ให้เล่นเสียงก่อนผู้ใช้แตะหน้าเว็บ	เพิ่มปุ่ม Tap to enable sound และการสั่น
คำขอสถานะบางครั้งหาย	คำขอมาตอนตัวส่งกำลังส่งเฟรมก่อนหน้า	ใช้ธง status_req ค้างไว้จนตัวส่งว่าง
เว็บยกเลิก Follow เองทันทีหลังกด	เฟรมสถานะเก่าที่ยังไม่มีตำแหน่งใหม่มาถึงก่อน	ESP จำเวลาที่ส่งคำสั่ง และยกเลิกเฉพาะเมื่อเฟรมใหม่กว่าคำสั่งบอกว่าไม่มีตำแหน่ง
ที่นั่งรองรับได้แค่ 3 ที่	ใช้สวิตช์ 1 ตัวต่อ 1 ที่	เปลี่ยนเป็นเลขฐานสอง 3 บิต รองรับ 7 ที่
ไฟชั้นสีเขียวหรี่	LED เขียว Vf สูงใกล้ 3.3 V	ใช้ LED สีเหลืองแทนที่ชั้นสีเขียว

12. ทรัพยากรที่ใช้บน FPGA

ผลจาก Vivado หลัง place and route ที่สัญญาณนาฬิกา 50 MHz (คาบ 20 ns)

ตารางที่ 14 ทรัพยากรและผล timing

ทรัพยากร	บอร์ด 1	บอร์ด 2	มีทั้งหมด
Slice LUTs	638 (7.98%)	123 (1.54%)	8,000
Slice Registers	693 (4.33%)	119 (0.74%)	16,000
DSP	1 (5.00%)	0	20
Block RAM	0	0	10

ทรัพยากร	บอร์ด 1	บอร์ด 2	มีทั้งหมด
ขา I/O	46 (46%)	27 (27%)	100
BUFG (สัญญาณนาฬิกา)	1	1	16
Worst Negative Slack	11.016 ns	13.063 ns	ต้องมากกว่า 0

ทั้งสองบอร์ดผ่าน timing ทุกเส้นทาง บอร์ด 1 มี slack 11 ns แปลว่าเส้นทางที่ช้าที่สุดใช้ราว 9 ns จึงทำงานได้ถึงราว 110 MHz ตัวคูณสำหรับแปลงอุณหภูมิถูกสังเคราะห์เป็น DSP 1 ตัว และใช้ทรัพยากรไม่ถึง 10% ของชิป

โค้ด VHDL

บทนี้แสดงโค้ดของ entity หลักที่ใช้ซ้ำในทั้งสองบอร์ด และส่วนสำคัญของ top entity โค้ดฉบับเต็มอยู่ในไฟล์ board1.vhd และ board2.vhd

1. uart_rx (ตัวรับ UART)

```
entity uart_rx is
  generic (CLKS_PER_BIT : integer := 434);   -- 50 MHz / 115200
  port (
    clk   : in  std_logic;
    rx    : in  std_logic;
    data  : out std_logic_vector(7 downto 0);
    valid : out std_logic
  );
end uart_rx;


architecture rtl of uart_rx is
  type state_t is (IDLE, START, DATA_BITS, STOP);
  signal state : state_t := IDLE;
  signal rx_s1, rx_s2 : std_logic := '1';
  signal cnt : integer range 0 to CLKS_PER_BIT - 1 := 0;
  signal idx : integer range 0 to 7 := 0;
  signal sh  : std_logic_vector(7 downto 0) := (others => '0');
begin
  process(clk)
  begin
    if rising_edge(clk) then
      rx_s1 <= rx;
      rx_s2 <= rx_s1;
      valid <= '0';


      case state is
        when IDLE =>
          cnt <= 0;
          idx <= 0;
          if rx_s2 = '0' then
            state <= START;
          end if;


        when START =>
          if cnt = CLKS_PER_BIT / 2 - 1 then
            cnt <= 0;
            if rx_s2 = '0' then
              state <= DATA_BITS;
```

```
            else
              state <= IDLE;
            end if;
          else
            cnt <= cnt + 1;
          end if;


        when DATA_BITS =>
          if cnt = CLKS_PER_BIT - 1 then
            cnt <= 0;
            sh(idx) <= rx_s2;
            if idx = 7 then
              state <= STOP;
            else
              idx <= idx + 1;
            end if;
          else
            cnt <= cnt + 1;
          end if;


        when STOP =>
          if cnt = CLKS_PER_BIT - 1 then
            cnt <= 0;
            if rx_s2 = '1' then
              data  <= sh;
              valid <= '1';
            end if;
            state <= IDLE;
          else
            cnt <= cnt + 1;
          end if;
      end case;
    end if;
  end process;
end rtl;
```

2. uart_frame_tx (ตัวส่งเฟรม UART)

```
entity uart_frame_tx is
  generic (
    CLKS_PER_BIT : integer := 434;
    N            : integer := 5
  );
  port (
    clk   : in  std_logic;
    start : in  std_logic;
    data  : in  std_logic_vector(8 * N - 1 downto 0);
```

```
    busy  : out std_logic;
    tx    : out std_logic
  );
end uart_frame_tx;


architecture rtl of uart_frame_tx is
  signal sh   : std_logic_vector(10 * N - 1 downto 0) := (others => '1');
  signal cnt  : integer range 0 to CLKS_PER_BIT - 1 := 0;
  signal bits : integer range 0 to 10 * N := 0;
begin
  tx   <= sh(0);
  busy <= '1' when bits /= 0 else '0';


  process(clk)
  begin
    if rising_edge(clk) then
      if bits = 0 then
        if start = '1' then
          for i in 0 to N - 1 loop
            sh(10 * i + 9 downto 10 * i) <= '1' & data(8 * i + 7 downto 8 * i)
& '0';
          end loop;
          bits <= 10 * N;
          cnt  <= 0;
        end if;
      elsif cnt = CLKS_PER_BIT - 1 then
        cnt  <= 0;
        sh   <= '1' & sh(10 * N - 1 downto 1);
        bits <= bits - 1;
      else
        cnt <= cnt + 1;
      end if;
    end if;
  end process;
end rtl;
```

3. debounce (กรองสัญญาณปุ่ม)

```
entity debounce is
  generic (
    STABLE_CLKS : integer   := 500_000;   -- 10 ms
    INIT        : std_logic := '0'
  );
  port (
    clk  : in  std_logic;
    din  : in  std_logic;
    dout : out std_logic
```

```
  );
end debounce;


architecture rtl of debounce is
  signal s1, s2 : std_logic := INIT;
  signal state  : std_logic := INIT;
  signal cnt    : integer range 0 to STABLE_CLKS := 0;
begin
  dout <= state;


  process(clk)
  begin
    if rising_edge(clk) then
      s1 <= din;
      s2 <= s1;
      if s2 = state then
        cnt <= 0;
      elsif cnt = STABLE_CLKS then
        state <= s2;
        cnt   <= 0;
      else
        cnt <= cnt + 1;
      end if;
    end if;
  end process;
end rtl;
```

4. melody (เพลงตอน Follow)

```
entity melody is
  port (
    clk   : in  std_logic;
    start : in  std_logic;
    tone  : out std_logic;
    busy  : out std_logic
  );
end melody;


architecture rtl of melody is
  constant MS  : integer := 50_000;    -- 1 ms ที่ 50 MHz
  constant GAP : integer := 25;        -- เงียบท้ายโน้ต 25 ms ให้โน้ตซ้ำแยกกันได้ยิน


  -- half = จำนวน clock ครึ่งคาบ = 50 MHz / (2 x ความถี่) / 0 = พัก
```

```
  constant R  : integer := 0;         -- พัก
  constant G5 : integer := 31_888;    --  784.0 Hz
  constant C6 : integer := 23_889;    -- 1046.5 Hz
  constant D6 : integer := 21_282;    -- 1174.7 Hz
  constant E6 : integer := 18_961;    -- 1318.5 Hz
  constant F6 : integer := 17_897;    -- 1396.9 Hz
  constant G6 : integer := 15_944;    -- 1568.0 Hz
  constant A6 : integer := 14_205;    -- 1760.0 Hz


  type note_t is record
    half : integer range 0 to 32_767;
    ms   : integer range 1 to 2_000;
  end record;
  type song_t is array (natural range <>) of note_t;
  constant SONG : song_t := (
    (E6, 125), (E6, 125), (R, 125), (E6, 125),
    (R, 125), (C6, 125), (E6, 250), (G6, 250),
    (R, 250), (G5, 250)
  );


  signal playing : std_logic := '0';
  signal idx     : integer range 0 to SONG'high := 0;
  signal ms_cnt  : integer range 0 to MS - 1 := 0;
  signal ms_left : integer range 0 to 2_000 := 0;
  signal tcnt    : integer range 0 to 32_767 := 0;
  signal sq      : std_logic := '0';
begin
  busy <= playing;
  tone <= sq when playing = '1' and ms_left > GAP and SONG(idx).half /= 0 else
'0';


  process(clk)
  begin
    if rising_edge(clk) then
      if start = '1' then
        playing <= '1';
        idx     <= 0;
        ms_cnt  <= 0;
        ms_left <= SONG(0).ms;
        tcnt    <= 0;
        sq      <= '0';
      elsif playing = '1' then
     -- สร้างคลื่นสี่เหลี่ยมตามความถี่ของโน้ต
        if tcnt >= SONG(idx).half - 1 then
          tcnt <= 0;
          sq   <= not sq;
        else
```

```
          tcnt <= tcnt + 1;
        end if;


    -- นับเวลาโน้ตทีละ 1 ms
        if ms_cnt = MS - 1 then
          ms_cnt <= 0;
          if ms_left <= 1 then
            if idx = SONG'high then
              playing <= '0';
            else
              idx     <= idx + 1;
              ms_left <= SONG(idx + 1).ms;
              tcnt    <= 0;
            end if;
          else
            ms_left <= ms_left - 1;
          end if;
        else
          ms_cnt <= ms_cnt + 1;
        end if;
      end if;
    end if;
  end process;
end rtl;
```

5. ตัวถอดรหัสไฟโซนและไฟชั้น (board2_top)

```
  -- decoder ไฟโซน: เลือก 1 ดวงจาก 3 ดวง ค่าอื่นดับหมด
  with loc select zone_led <=
    "001" when x"11" | x"12" | x"13",   -- โซน 1
    "010" when x"21" | x"22" | x"23",   -- โซน 2
    "100" when x"31" | x"32" | x"33",   -- โซน 3
    "000" when others;


  -- decoder ไฟชั้น: เลือก LED 1 ดวงจาก 9 ดวง ค่าอื่นดับหมด
  with loc select shelf_led <=
    "000000001" when x"11",   -- โซนแดง   ชั้นแดง
    "000000010" when x"12",   -- โซนแดง   ชั้นน้ำเงิน
    "000000100" when x"13",   -- โซนแดง   ชั้นเขียว
    "000001000" when x"21",   -- โซนน้ำเงิน ชั้นแดง
```

```
    "000010000" when x"22",
    "000100000" when x"23",
    "001000000" when x"31",    -- โซนเขียว  ชั้นแดง
    "010000000" when x"32",
    "100000000" when x"33",
    "000000000" when others;
```

ผลการทดลองและการทดสอบ

1. แนวทางการทดสอบ

- **ทดสอบด้วยเครื่องมือ:** สังเคราะห์และตรวจ timing ทุกครั้งที่แก้โค้ด, ตรวจ syntax ของ JavaScript, คอมไพล์โปรแกรม ESP8266, เปิดหน้าเว็บใน Chrome ที่ความกว้างจอคอมและจอมือถือ
- **ทดสอบระดับโมดูลบนบอร์ด:** ใช้ 7-segment, LCD และ LED แสดงค่าภายในของแต่ละ entity เช่นค่าดิบ ADC และจำนวนเฟรมเสีย
- **ทดสอบรวมระบบ:** ทดสอบตามสถานการณ์ใช้งานจริงบนโมเดล ตามตารางที่ 17

2. ผลการทดสอบด้วยเครื่องมือ

ตารางที่ 15 ผลการทดสอบด้วยเครื่องมือ

รายการ	ผล
สังเคราะห์และสร้าง bitstream บอร์ด 1, บอร์ด 2, esp_upload	0 Errors ทั้ง 3 ตัว
Timing ที่ 50 MHz	ผ่าน (WNS 11.016 ns และ 13.063 ns)
คอมไพล์โปรแกรม ESP8266	ผ่าน, ใช้ RAM 36%, flash 30%
ตรวจ syntax JavaScript ของหน้าเว็บ (Node.js)	ผ่าน
แสดงผลหน้า Seat ที่ความกว้าง 1200 px และ 360 px	แถบที่นั่ง 7 ช่องแสดงครบ ไม่ล้นจอ

3. ผลการทดสอบบนบอร์ดที่ยืนยันแล้ว

ตารางที่ 16 ผลการทดสอบบนบอร์ดที่บันทึกไว้

รายการ	วิธีทดสอบ	ผล
ESP8266 ⟶ คอมพิวเตอร์ ผ่าน FPGA	SW8 = 1, รีเซ็ต ESP, ดู Serial Monitor ที่ COM3	เห็น ets Jan 8 2013,rst cause:2 (24 ต.ค. 2569)
คอมพิวเตอร์ ⟶ ESP8266 ผ่าน FPGA	อัปโหลดโปรแกรม ESP จาก Arduino IDE	อัปโหลดสำเร็จ (24 ต.ค. 2569)

รายการ	วิธีทดสอบ	ผล
Buzzer เตือนควัน	SW9 = 1 และพ่นควันที่ MQ-2	มีเสียงดังเป็นจังหวะ (24 ต.ค. 2569)
เสียงเตือนบนเว็บ	หน้า Smoke แตะกล่องสถานะบนมือถือ Android	ได้ยินเสียง (25 ต.ค. 2569)
เขียน Flash บอร์ด 1	upload_board1_prom.bat	Erase และ Write ครบ 100%

4. ตารางทดสอบรวมระบบ

ใช้ตารางนี้ทดสอบก่อนนำเสนอ และทำเครื่องหมาย ✓ ในช่องผ่านเมื่อได้ผลตามที่ต้องได้

ตารางที่ 17 กรณีทดสอบรวมระบบ

กรณีทดสอบ	ขั้นตอน	ผลที่ต้องได้	ผ่าน
ที่นั่ง	SW2 = 1, SW4 = 1	LCD Used 3 / Free 4, เว็บช่อง 1–3 เต็ม	☐
ที่นั่งเต็ม	SW2 SW4 SW5 = 1	LCD Used 7 / Free 0	☐
ไฟ Auto	บังแสงที่ LDR	ไฟห้อง 3 ดวงติด เว็บขึ้น On	☐
สลับโหมด	กด SW18	โหมดสลับ เว็บหน้า Light เปลี่ยนตาม	☐
ไฟ Manual	โหมด Manual กด SW14 หรือปุ่มบนเว็บ	ไฟสลับติด/ดับ	☐
พัดลม	กด SW16 หรือเลือก On/Off บนเว็บ	พัดลมหมุน/หยุด สถานะตรงกัน	☐
แอร์	อุ่น LM35 ด้วยมือ ดู 7-segment	อุณหภูมิแอร์ลดลงทีละ 0.5 °C ทุก 2 วินาที	☐
ควัน	พ่นควันที่ MQ-2	buzzer ดังเป็นจังหวะ, LCD FIRE ALARM, เว็บเตือนเต็มจอพร้อมไซเรน	☐
ควันหาย	รอให้ควันจาง	buzzer หยุด, LCD กลับเป็นที่นั่ง, เว็บรอกด OK	☐
ค้นหา	เปิดหน้า Search พิมพ์ 3	ตอนแรกเห็น 9 เล่ม พิมพ์แล้วเหลือ Book No.3	☐
Follow	กด Follow ที่ Book No.5	เพลงเล่น, ไฟโซน 2 + ชั้น 2 ติด, 7-seg บอร์ด 2 แสดง 2-2	☐
ยกเลิก Follow	กด Follow ซ้ำ	ไฟดับ, บอร์ด 2 แสดง ----, ไม่มีเพลง	☐
สายหลุด	ถอดสาย C14 ขณะ Follow	ไฟดับภายใน 0.5 วินาที	☐
คุณภาพสาย	SW2 บอร์ด 2 = 1 ขณะ Follow	E000 ค้าง จุดขวาสุดกะพริบสม่ำเสมอ	☐

5. ค่าที่ต้องปรับเทียบก่อนนำเสนอ

- TEMP_SCALE ใน board1.vhd (ค่าปัจจุบัน 250): ตั้ง SW7-SW6 = 10 แล้วเทียบกับเทอร์โมมิเตอร์
- เกณฑ์ LDR (2100 / 2300): ตั้ง SW6 = 1 อ่านค่าดิบตอนห้องสว่างและมืดในสถานที่นำเสนอ
- ความไวของ MQ-2: ปรับตัวต้านทานปรับค่าบนโมดูลให้ไม่เตือนในอากาศปกติ

สรุปและข้อเสนอแนะ

1. สิ่งที่ประสบความสำเร็จ

Smart Library ใช้ FPGA 2 บอร์ดควบคุมระบบห้องสมุด 6 ระบบพร้อมกัน ออกแบบแบบ Top-Down จากระดับระบบลงถึง entity ของ VHDL 8 ชนิด และประยุกต์ความรู้ในรายวิชาครบทั้ง FSM ตัวนับ ตัวแบ่งความถี่ decoder วงจรบวกลบ การแสดงผล 7-segment และ LCD และการสื่อสาร SPI และ UART ทั้งสองบอร์ดใช้ทรัพยากรไม่ถึง 10% และผ่าน timing ที่ 50 MHz ด้วย slack มากกว่า 11 ns

2. ข้อจำกัดและสิ่งที่ยังต้องปรับปรุง

- แอร์เป็นการจำลองบน 7-segment ไม่ได้ควบคุมแอร์จริง
- จำนวนที่นั่งมาจากสวิตช์ ไม่ได้มาจากเซนเซอร์ที่โต๊ะ
- จอ LCD แสดงได้เฉพาะอักษรภาษาอังกฤษ
- ESP8266 รองรับเครื่องเชื่อมต่อสูงสุด 8 เครื่อง และตอบคำขอทีละรายการ

3. ข้อเสนอแนะ

- ใช้เซนเซอร์แรงกดหรืออินฟราเรดที่แต่ละโต๊ะ แทนสวิตช์เลขฐานสอง
- ปรับความเร็วพัดลมด้วย PWM แทนการเปิด-ปิด
- ใช้ RFID ติดหนังสือเพื่อตรวจว่าหนังสืออยู่บนชั้นจริง
- เพิ่มบอร์ดไฟชั้นได้ด้วยโปรโตคอลเดิม เพราะบอร์ด 2 รับข้อมูลทางเดียว

แหล่งจัดเก็บข้อมูล

วีดีทัศน์แนะนำชิ้นงาน: CE_KMITL_Digital_2569_SmartLibrary

ลิงก์ย่อ: [ใส่ลิงก์ย่อหลังอัปโหลดวีดีทัศน์]

QR Code: [ใส่ QR Code ของลิงก์วีดีทัศน์]

ไฟล์โครงงานทั้งหมด (VHDL, โปรแกรม ESP8266, หน้าเว็บ, ไฟล์ .bit และเอกสาร) อยู่ในโฟลเดอร์ Project ของสื่อบันทึกข้อมูลที่ส่งพร้อมชิ้นงาน

ภาคผนวก ก ตารางขาบอร์ด 1 และบอร์ด 2

ตารางที่ 18 ขาของบอร์ด 1 (board1_top)

port	ทิศ	ขา	บนบอร์ด
clk	in	H11	OSC 50 MHz
usb_rx / usb_tx	in / out	G1 / F2	UART_RXD / UART_TXD
esp_tx / esp_rx	in / out	A10 / A12	WIFI_RXD / WIFI_TXD
btn_light / btn_fan / btn_mode	in	J14 / J13 / J12	ปุ่ม SW14 / SW16 / SW18
sw_seat[0..2]	in	K11, M11, N14	SW2, SW4, SW5
sw_view[0..1]	in	P12, N10	SW6, SW7
sw_prog / sw_buzz	in	P10 / M10	SW8 / SW9
smoke_do	in	L14	J5 (MQ-2 DO)
adc_dout / adc_sck / adc_din / adc_cs	in / out	G4 / D1 / C1 / F4	MCP3208
room_led / fan / buzzer	out	M13 / F11 / B14	J5 / J5 / BUZZER
seg[0..6], dp	out	L3 P4 P2 M3 M1 J4 K4, J2	SEG_A..G, DP (active-low)
digit[0..3]	out	H1 H2 H3 H4	DIGIT4..DIGIT1
lcd_rs / lcd_en	out	M5 / P5	LCD
lcd_data[0..7]	out	M4 L5 P11 N11 P13 M14 M12 K12	LCD_D0..D7
link_tx	out	C14	J5 ไปบอร์ด 2

ตารางที่ 19 ขาของบอร์ด 2 (board2_top)

port	ทิศ	ขา	บนบอร์ด
clk	in	H11	OSC 50 MHz
link_rx	in	C14	J5 จากบอร์ด 1
sw_debug	in	K11	SW2
zone_led[0..2]	out	L14, M13, H14	ไฟโซน แดง น้ำเงิน เขียว
shelf_led[0..2]	out	D14, E13, F13	โซนแดง: ชั้นแดง น้ำเงิน เขียว (LED เหลือง)
shelf_led[3..5]	out	F14, G14, D13	โซนน้ำเงิน: ชั้นแดง น้ำเงิน เขียว (LED เหลือง)

port	ทิศ	ขา	บนบอร์ด
shelf_led[6..8]	out	D12, E12, F12	โซนเขียว: ชั้นแดง น้ำเงิน เขียว (LED เหลือง)
seg, dp, digit	out	เหมือนบอร์ด 1	7-segment

ภาคผนวก ข รายการหนังสือและไฟล์ในโครงงาน

ตารางที่ 20 รายการหนังสือในระบบ

เล่ม	โซน	ชั้น	คงเหลือ
Book No.1	1 แดง	1 แดง	3
Book No.2	1 แดง	2 น้ำเงิน	1
Book No.3	1 แดง	3 เขียว	2
Book No.4	2 น้ำเงิน	1 แดง	0
Book No.5	2 น้ำเงิน	2 น้ำเงิน	2
Book No.6	2 น้ำเงิน	3 เขียว	1
Book No.7	3 เขียว	1 แดง	4
Book No.8	3 เขียว	2 น้ำเงิน	2
Book No.9	3 เขียว	3 เขียว	1

ตารางที่ 21 ไฟล์ในโครงงาน

ไฟล์	หน้าที่
board1.vhd	บอร์ด 1: uart_rx, uart_frame_tx, debounce, mcp3208, sevenseg, lcd16x2, melody, board1_top
board2.vhd	บอร์ด 2: uart_rx, sevenseg_loc, board2_top
esp_upload.vhd	วงจรส่งข้อมูลทะลุ USB ↔ ESP8266 สำหรับอัปโหลดโปรแกรม
fpga_build/	ไฟล์ขา XDC และสคริปต์ build ของ Vivado
smart_library/	โปรแกรม ESP8266 (.ino) และหน้าเว็บ (index_html.h)
upload_*.bat	สคริปต์โหลดไฟล์ลง FPGA แบบ SRAM และ Flash
SYSTEM.md, pins.md	เอกสารระบบ และบันทึกขาและผลทดสอบ